

Designing a Redux Store

Dynamic Web Applications · Week 11

Two ways round

THE USUAL WAY

- Build the components.
- Notice they need to share something.
- Fit a store around what you already built.
- Discover halfway that two pieces of state disagree.

TODAY

- Work out what the state is.
- Work out how it changes.
- Group it into slices.
- Then, and only then, open the editor.

This is how it is done when the app is not a toy.

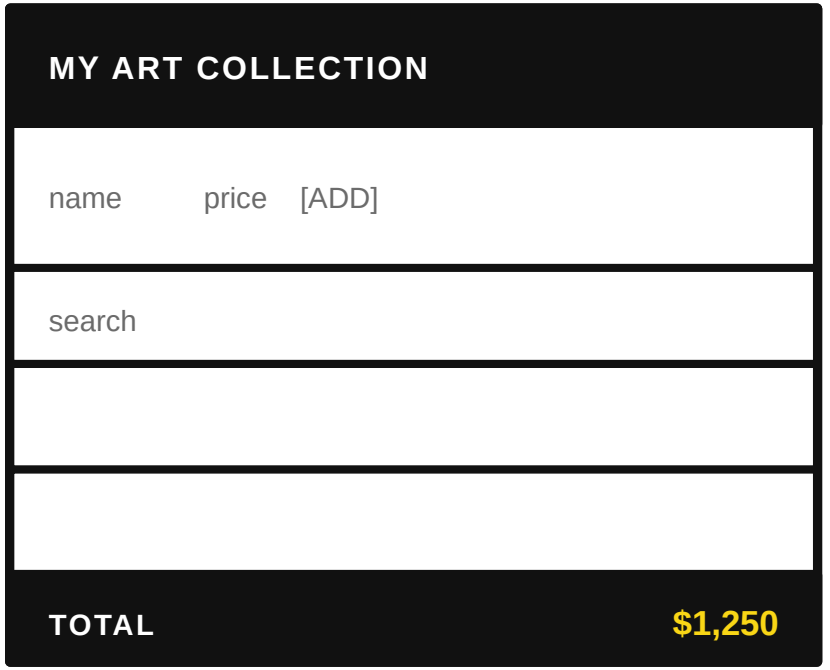
Four questions, every time

1. **What state exists in this app?**
2. **How does it change over time?**
3. **Which pieces belong together?**
4. **What can be worked out from the rest?**

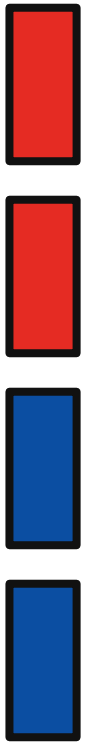
Answer all four and the store is written. The typing is transcription.

The app we are building

- Add a piece of art, with a name and a price
- Show the collection, and delete from it
- Search the collection by name
- Show the total value of whatever is currently showing



1. What state exists



the name being typed `a string`

the price being typed `a number`

the search term `a string`

the collection `an array of {id, name, price}`

Four things. Nothing about components, nothing about files.

2. How does it change

changeName	changePrice	changeSearchTerm	addArt	removeArt

Those are your reducers. You just wrote the store and have not opened an editor.

3. Which pieces belong together

formSlice

```
name
price
changeName
changePrice
```

adding a piece

artSlice

```
searchTerm
data
changeSearchTerm
addArt
removeArt
```

showing the collection

Two jobs, two slices. You could split further. Two is a judgement call, not a rule.

4. What can be worked out from the rest

the visible list	=	the collection, filtered by the search term
the total value	=	the visible list, added up

Neither of these is state. Both are questions you ask while rendering.



If you can calculate it, do not store it

Because two copies disagree

STORED

```
data: [
  {name, price, visible: true}
]
```

- Search for "mondrian".
- Add a Rothko while the search is on.
- It arrives with visible: true.
- It shows up in a filtered list. Wrong.

WORKED OUT

```
data.filter((art) =>
  art.name.includes(term)
)
```

- Nothing new is stored.
- So nothing new can go stale.
- No reducer has to remember to update it.
- It is right by construction.

The whole store, before any JSX

```
{
  art:  { searchTerm: "",  data: [] },
  form: { name: "",        price: 0  },
}
```

Every useSelector you write today starts from this tree.

Leave it on the board. The two-column table from question 3 matches it, column for column.

A slice cannot see another slice

artSlice

```
addArt(state, action) {
  state.form.name
}
```

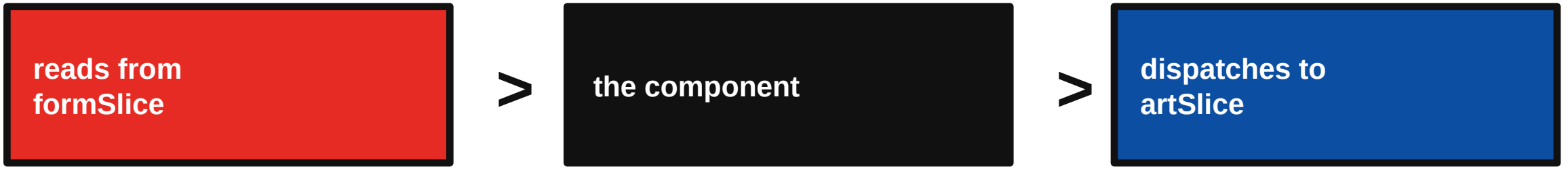
X

formSlice

```
name:  "Mondrian..."
price: 500
```

Inside a reducer, state is that slice and nothing else. There is no state.form to reach.

So the component is the bridge




```
const name  = useSelector((state) => state.form.name)
dispatch(addArt({name, price}))
```

The component can see both, because it is outside them. Write the payload shape down as a comment: it is the only contract the reducer will ever have.



Now open your laptops

What state exists. How it changes. What groups together. What can be worked out.

If you can calculate it, do not store it.